CPJudgeBench: Meta-Evaluation of LLM Judges for Constraint Programming through Solution-Space Semantics

Nastaran Moradzadeh Farid[1], Alireza Taghizadeh[2], Tommaso Di Noia[3], Yashar Deldjoo[3], Sara Shafiee[1],
[1]Technical University of Denmark, Denmark,
[2]Configit A/S, Denmark, [3]Politecnico di Bari, Italy.
Correspondence: nmofa@dtu.dk

Abstract

Large Language Models (LLMs) are increasingly used as evaluators for code generation, but existing judge benchmarks mainly target general-purpose programming. Constraint Programming (CP) requires a stricter notion of correctness: a model is correct only if it faithfully represents the intended solution space, not merely if it executes, preserves satisfiability, or returns one valid solution. We introduce **CPJudgeBench**, a benchmark for meta-evaluating LLM judges of CP modeling correctness through solver-supported solution-space semantics. It contains 522 solver-validated candidate models from 82 CP problem families in CPMpy and MiniZinc, labeled as equivalent, unsound, incomplete, mixed, or non-executable. Across our experiments, we meta-evaluate a total of 15 LLM judges through semantic-label, binary-correctness, and score-based protocols. Results show that fine-grained semantic judging remains difficult, especially for reference-free MiniZinc, while stronger reasoning- and code-oriented models perform more reliably. Score-based LLM judgments also align better with solution-space similarity than standard text or code metrics. The benchmark system is available at `https://github.com/Nastaran95/CPJudgeBench`.

1 Introduction

Constraint programming (CP) poses a distinctive challenge for evaluating generated code. A CP model specifies decision variables, domains, constraints, and, in optimization settings, an objective; a solver then searches for assignments satisfying the resulting specification (Nethercote et al., 2007; Lecoutre and Szczepanski, 2020; Guns, 2019). Correctness is therefore semantic rather than merely operational. Two models may be syntactically different yet represent the same feasible set, while an executable model can return a plausible assignment and still encode the wrong problem. A CP model is correct only when it admits the intended feasible assignments and excludes invalid ones; with objectives, it must also preserve the intended optimality structure.

This semantic requirement exposes a gap in current LLM-based evaluation. General-purpose (GP) code metrics and execution benchmarks approximate correctness through reference similarity, tests, or input–output behavior (Ren et al., 2020; Zhou et al., 2023; Chen et al., 2021; Zhuo et al., 2025), while LLM-as-a-judge methods assess generated code or responses directly but are commonly meta-evaluated using human labels, test suites, or proxy model judgments (Zhuo, 2024; Tong and Zhang, 2024; Zhao et al., 2025; Tan et al., 2024; Bavaresco et al., 2025; Jiang et al., 2025). In parallel, recent CP-generation benchmarks show that LLMs can produce models in MiniZinc, PyCSP3, CPMpy, and related systems, but they mainly evaluate generation through executability, satisfiability status, single-solution validity, or objective values (Michailidis et al., 2025, 2026; Song and Cohen, 2025; Shi et al., 2025). These signals are valuable, but they can miss CP-specific errors: a candidate may share the reference SAT/UNSAT status while admitting invalid assignments, return one valid solution while excluding others, or use auxiliary variables and alternative encodings that make syntactic comparison and raw assignment matching inappropriate.

Such errors are consequential in product configuration and engineering design, where CP models represent feasible combinations of features, components, or parameter values (Shafiee et al., 2018, 2021; Shawky et al., 2026; Pan et al., 2025; Zhang et al., 2025; Fu et al., 2024; Penco et al., 2025). Unsound models may recommend infeasible variants, whereas incomplete models may remove valid choices from the design space; Appendix C provides additional motivation for these semantic error types. Evaluating CP judges therefore requires

Table 1: Positioning CPJUDGEBENCH against representative code-judge and CP-generation benchmarks. ● = primary focus; △ = partial or indirect coverage; – = not addressed or not applicable. CPJUDGEBENCH is distinguished by combining CP-specific model judging with a solver-supported solution-space oracle for automated meta-evaluation.

Domain	Work	LLM role		Evaluation signal				Judge protocol			Meta-eval oracle			
		Gen.	Judge	Exec./tests	Status	Single sol.	Sol. space	Point	Pair	Rank/score	Human/expert	Test-case	LLM/agent	Solver/auto
GP	HumanEval (Chen et al., 2021)	●	–	●	–	△	–	–	–	–	–	–	–	–
GP	BigCodeBench (Zhuo et al., 2025)	●	–	●	–	△	–	–	–	–	–	–	–	–
GP	ICE-Score (Zhuo, 2024)	–	●	△	–	△	–	●	△	△	●	△	–	–
GP	CodeJudge (Tong and Zhang, 2024)	–	●	△	–	△	–	●	△	△	△	●	–	–
GP	CodeJudge-Eval (Zhao et al., 2025)	–	●	△	–	△	–	●	–	△	●	–	–	–
GP	JudgeBench (Tan et al., 2024)	–	●	–	–	△	–	●	●	△	–	–	●	●
GP	JUDGE-BENCH (Bavaresco et al., 2025)	–	●	–	–	△	–	●	–	△	●	–	–	–
GP	CodeJudgeBench (Jiang et al., 2025)	●	●	△	–	△	–	●	●	△	–	●	–	●
CP	CP-Bench / DCP-Bench-Open (Michailidis et al., 2025, 2026)	●	–	●	●	●	△	–	–	–	–	–	–	–
CP	CPEVAL (Song and Cohen, 2025)	●	–	●	△	●	–	–	–	–	–	–	–	–
CP	ConstraintLLM / IndusCP (Shi et al., 2025)	●	–	●	△	●	–	–	–	–	–	–	–	–
CP	**CPJUDGEBENCH** (ours)	●	●	●	△	△	●	●	–	●	–	–	–	●

labels defined over solution-space relations, not only over surface similarity, solver status, or a single solver output. A practical obstacle is that existing CP datasets provide problem descriptions, reference models, and execution-oriented signals, but not controlled candidate models with solver-validated semantic relations to a reference; a CP judge benchmark therefore needs both diverse candidate errors and an independent mechanism for validating their semantic labels.

We address this need with CPJUDGEBENCH, a benchmark for meta-evaluating LLM judges for CP modeling correctness. CPJUDGEBENCH separates two roles that are often conflated: a generation-side model G_ϕ constructs candidate CP models, while an evaluation-side model J_θ judges them. Candidate labels are not assigned by the judge or by the generator. Instead, they are produced by a solver-supported oracle that compares projected candidate and reference solution spaces over shared semantic decision variables. This construction yields five semantic labels—EQ, UNS, INC, MIX, and NONEXEC—as well as graded solution-space similarity scores. Appendix A illustrates these labels using a domino-tiling example, and Figure 1 summarizes the two-stage pipeline used to generate, validate, and judge candidate models.

Related work and novelty. Table 1 positions existing code-evaluation, code-judge, and CP-generation benchmarks along the LLM role, evaluation signal, judge protocol, and meta-evaluation oracle axes. Across these axes, GP benchmarks cover both generation and LLM judging, and judge-centric GP work studies pointwise, pairwise, or score-based evaluation against human labels, tests, or proxy oracles (Chen et al., 2021; Zhuo et al., 2025; Ren et al., 2020; Zhou et al., 2023; Zhuo, 2024; Tong and Zhang, 2024; Zhao et al., 2025; Tan et al., 2024; Bavaresco et al., 2025; Jiang et al., 2025). However, their evaluation signals are generally incomplete approximations of program semantics. CP benchmarks, by contrast, exploit solver execution but have mainly treated LLMs as generators, with emphasis on executability, status, and solution-level checks rather than judge meta-evaluation (Michailidis et al., 2025, 2026; Song and Cohen, 2025; Shi et al., 2025). CPJUDGEBENCH fills this gap by combining CP-specific semantic oracles, controlled candidate generation, and automated meta-evaluation of LLM judges. For finite tractable instances, solvers can enumerate feasible assignments, project them onto shared semantic variables, and compare candidate and reference solution spaces; CPJUDGEBENCH uses this property to evaluate whether LLM judges recover CP semantics rather than merely matching tests, surface similarity, or another model.

Contributions. This paper makes three contributions:

1. We formalize LLM-as-a-judge evaluation for CP modeling correctness by distinguishing problem descriptions, reference models, candidate models, semantic projection variables, solver-supported oracles, and judge outputs (§2).

2. We introduce CPJUDGEBENCH, a benchmark of 522 solver-validated candidate models from 82 CP problem families in CPMpy and MiniZinc, with multi-class semantic labels, solution-space similarity scores, prompts, solver logs, and validation metadata.

3. We evaluate LLM-as-a-judge methods under two research questions: generator-level reliability of the construction pipeline (RQ1), and CP semantic judge meta-evaluation (RQ2), including multi-class, binary, score-based, model-capability, and cost trade-off analyses across 15 judges.

2 CPJUDGEBENCH: Formalism and Methodology

CPJudgeBench evaluates whether LLMs can judge the semantic correctness of CP models. Figure 1 organizes the benchmark into two stages. Stage 1 constructs a solver-validated candidate dataset from a problem description, reference model, target language, and requested semantic label. Stage 2 uses the validated candidates for LLM-as-a-judge meta-evaluation, comparing each judge prediction against the oracle label or score. A more detailed representation of the framework is provided in Appendix B.

Objects and Notation. A problem family q consists of a natural-language description d_q, a reference CP model $M_q^\star$, a CP language ℓ, and semantic decision variables V_q. The variables in V_q define the comparison space used by the oracle, allowing candidate and reference models to differ in names, auxiliary variables, or encodings while still being compared semantically.

Definition 1 (CP model and feasible assignments)**.** *A constraint satisfaction model is $M = (X, D, C)$, where $X = \{x_1, \ldots, x_n\}$ is a set of variables, $D = \{D_1, \ldots, D_n\}$ gives their domains, and C is a set of constraints. A complete assignment a is feasible for M if every x_i is assigned a value in D_i and all constraints in C are satisfied. We write $\mathrm{Sol}(M)$ for the feasible assignments of M.*

For semantic variables V, the projected solution space is $S_V(M) = \{\mathrm{proj}_V(a) \mid a \in \mathrm{Sol}(M)\}$. For problem family q, we write $S^\star = S_{V_q}(M_q^\star)$ and $S^c = S_{V_q}(M^c)$, where M^c is a candidate model. In Figure 3, V_q can encode horizontal and vertical domino placements: an overlapping tiling appears in $S^c \setminus S^\star$, whereas a missing valid tiling appears in $S^\star \setminus S^c$. For optimization problems, the present benchmark compares feasible-space semantics; objective-aware comparison can be added by replacing $S_V(M)$ with projected optimal assignments.

Solution-Space Oracle. The oracle first checks whether a candidate can be parsed, compiled, and solved under the validation protocol. Non-executable candidates receive NONEXEC. Executable candidates are compared to the reference through false positives $\mathrm{FP} = S^c \setminus S^\star$ and false negatives $\mathrm{FN} = S^\star \setminus S^c$. False positives are assignments admitted by the candidate but not by the reference; false negatives are reference-valid assignments excluded by the candidate.

Definition 2 (Semantic correctness labels)**.** *For an executable candidate M^c, the oracle label $y^\star$ is*

$$y^\star = \begin{cases} \textsc{Eq}, & \mathrm{FP} = \varnothing \text{ and } \mathrm{FN} = \varnothing, \\ \textsc{Uns}, & \mathrm{FP} \neq \varnothing \text{ and } \mathrm{FN} = \varnothing, \\ \textsc{Inc}, & \mathrm{FP} = \varnothing \text{ and } \mathrm{FN} \neq \varnothing, \\ \textsc{Mix}, & \mathrm{FP} \neq \varnothing \text{ and } \mathrm{FN} \neq \varnothing. \end{cases}$$

If M^c fails parsing, compilation, or solving, then $y^\star = \textsc{NonExec}$.

The labels correspond to Figure 3: EQ denotes semantic equivalence; UNS denotes a complete but unsound candidate; INC denotes a sound but incomplete candidate; and MIX denotes both error types. For executable candidates, the oracle also computes precision $P = |S^c \cap S^\star|/|S^c|$, recall $R = |S^c \cap S^\star|/|S^\star|$, and the similarity score $s^\star = 1$ when $S^c = S^\star$ and $s^\star = 2PR/(P+R)$ otherwise, with zero-valued precision or recall when a denominator is empty in a non-equivalent case. Enumeration is restricted to instances satisfying the benchmark resource limits.

2.1 Stage 1: Candidate Generation and Solver-Based Validation

Stage 1 constructs candidate models whose final labels are solver validated. For each problem family q, target language ℓ, and requested label y_{tgt}, the generator receives a label-specific instruction $\gamma_{y_{\mathrm{tgt}}}$. At attempt t, the prompt is $p_t^G =$

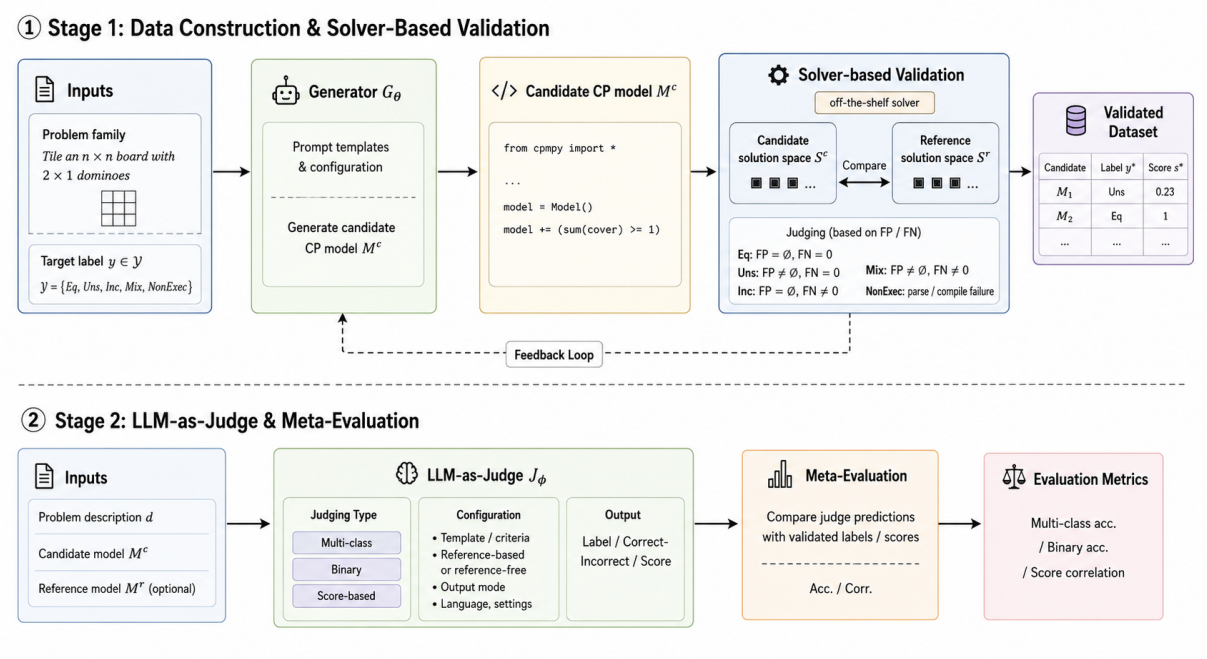

Figure 1: Overview of CPJUDGEBENCH. Stage 1 constructs solver-validated candidates by generating CP models for a target semantic label and validating them through projected solution-space comparison against a predefined reference model. The oracle assigns semantic labels $y^\star$ and similarity scores $s^\star$. Stage 2 meta-evaluates LLM judges by comparing their predicted labels, binary decisions, or scores against the solver-supported oracle.

$\mathcal{P}^G(d_q, M_q^\star, \ell, \gamma_{y_{\text{tgt}}}, h_t)$, where h_t contains feedback from earlier validation attempts and $h_1 = \varnothing$; the generator returns $M_t^c = G_\phi(p_t^G)$, with $t \le T$.

The requested label guides generation but is not the benchmark label. Each candidate is passed to the solver-supported oracle, $(y_t^\star, s_t^\star, w_t) = \mathcal{O}(M_t^c, M_q^\star, V_q)$, where w_t may include false-positive or false-negative witnesses. If $y_t^\star = y_{\text{tgt}}$, the candidate is retained; otherwise the feedback is added to h_{t+1}, and generation continues until a match is obtained or the maximum number of attempts T is reached. The validated dataset contains items $z_i = (d_i, M_i^\star, M_i^c, V_i, \ell_i, y_i^\star, s_i^\star, w_i)$.

This loop is central to RQ1 because it measures whether label-conditioned generation can populate semantic classes under solver validation, rather than assuming that the requested label is correct. For example, an attempted unsound domino-tiling model is retained as UNS only if the solver confirms that it admits extra invalid tilings and excludes no valid reference tilings.

2.2 Stage 2: LLM-as-a-Judge Meta-Evaluation

Stage 2 evaluates whether an LLM judge can recover oracle semantics. A judging setting τ specifies the prompt template, reference-free or reference-based context, and output format. The prompt is $p_i^J = \mathcal{P}_\tau^J(d_i, M_i^c, r_i)$, with $r_i \in \{\varnothing, M_i^\star\}$, and the judge output is $J_\theta(p_i^J) = (\hat{o}_i, e_i)$, where e_i is an optional explanation and $\hat{o}_i$ is one of three output types: a semantic label $\hat{y}_i$, binary decision $\hat{c}_i$, or similarity score $\hat{s}_i$.

Definition 3 (LLM-as-a-judge meta-evaluation for CP)**.** *Given a validated candidate z_i, judge J_θ, and oracle output $(y_i^\star, s_i^\star)$, CP meta-evaluation measures the agreement between the judge prediction and the oracle: $\mathcal{M}(J_\theta(p_i^J), (y_i^\star, s_i^\star))$. The oracle is independent of the judge and derived from projected solution-space comparison, so the evaluation measures recognition of CP semantics rather than agreement with another model or surface metric.*

For multi-class judging, $\hat{y}_i \in \mathcal{Y} = \{\text{EQ}, \text{UNS}, \text{INC}, \text{MIX}, \text{NONEXEC}\}$ and $\text{Acc}_{\text{multi}} = N^{-1} \sum_i \mathbb{I}[\hat{y}_i = y_i^\star]$. For binary judging, $c_i^\star = \mathbb{I}[y_i^\star = \text{EQ}]$, $\hat{c}_i \in \{0, 1\}$, and $\text{Acc}_{\text{bin}} = N^{-1} \sum_i \mathbb{I}[\hat{c}_i = c_i^\star]$. Binary evaluation is useful but cannot distinguish unsoundness, incompleteness, and mixed errors. For score-based judging, $\hat{s}_i$ is compared with $s_i^\star$ using Kendall's τ, Pearson's r, and Spearman's ρ. RQ2a uses these formats to test judge effectiveness; RQ2b analyzes how performance varies with model capability, specialization, and cost.

3 Experimental Setup and Benchmark Statistics

Dataset and candidate generation. We use 82 problem families from DCP-Bench-Open (Michailidis et al., 2026). These families provide natural-language descriptions, executable CPMpy reference models, predefined semantic decision variables, and parameterized instances. These elements support the projection-based oracle defined in § 2, although the generation-and-validation pipeline itself is dataset-agnostic and can be applied to any executable CP benchmark with suitable semantic variables. We use a fixed generator, GPT-5.4-mini, to avoid conflating generator variation with judge performance. Candidate models are produced with the label-conditioned Stage 1 prompts described in Appendix F; the requested label guides generation but is never used as the final benchmark label. The oracle independently assigns the final labels, which are hidden from the judges.

Languages and solvers. We evaluate two CP languages: CPMpy, a Python-embedded modeling library, and MiniZinc, a dedicated CP modeling language. CPMpy models are solved with OR-Tools CP-SAT through the CPMpy backend, and MiniZinc models with Gecode through the MiniZinc driver. Enumeration over the semantic variables V_q is limited to 60 seconds and 1,000,000 assignments; candidates exceeding either limit are excluded from the retained benchmark to ensure complete solution-space comparison. Under these fixed solver configurations, the oracle records inspectable solution counts and false-positive and false-negative witnesses supporting each assigned label. Because the original references are provided in CPMpy, MiniZinc candidates require the generator to reinterpret the problem semantics in a different modeling language, which makes MiniZinc a more challenging construction and judging setting.

Judge models and settings. We evaluate 15 LLM judges spanning commercial and open-weight systems, general-purpose and code-specialized models, dense and mixture-of-experts architectures, and different reasoning and cost profiles. We test the semantic-label, binary-correctness, and score-based formats from §2.2. Semantic-label and binary formats are evaluated in reference-free and reference-based settings where applicable; score-based judging uses a reference-based rubric because the score is defined as similarity to the reference solution space. Each model produces one judgment per candidate and setting through OpenRouter at temperature 1.0. Responses are requested in JSON, parsed first as JSON and then, when needed, through fallback extraction, with raw responses retained for auditing. Full judge metadata and experiment-time pricing information are provided in Appendix D, and the judge prompt templates are listed in Appendix G.

Baselines and metrics. As reference-based baselines, we compute BLEU (Papineni et al., 2002), ROUGE-L (Lin, 2004), METEOR (Banerjee and Lavie, 2005), chrF (Popović, 2015), RUBY (Tran et al., 2019), CodeBLEU (Ren et al., 2020), and CodeBERTScore (Zhou et al., 2023). These metric baselines are evaluated only on CPMpy, using CPMpy candidates and CPMpy reference models, to avoid cross-language surface-similarity confounds. For semantic-label and binary judging, we report accuracy against the oracle labels $y^\star$ and collapsed correctness labels $c^\star = \mathbb{I}[y^\star = \text{EQ}]$, respectively. For score-based judging and metric baselines, we report Spearman's ρ, Kendall's τ, and Pearson's r against the oracle score $s^\star$. Binary results are used only as a coarse supplementary diagnostic because the collapsed classes are highly imbalanced; details are moved to Appendix H.

Benchmark statistics. Table 2 summarizes the final solver-validated dataset. CPJUDGEBENCH contains 522 candidate models from 82 problem families, covering all five oracle labels: EQ, UNS, INC, MIX, and NONEXEC. CPMpy has a more balanced label distribution, while MiniZinc contains a larger share of non-executable candidates. This reflects the additional difficulty of generating candidates in a dedicated CP language when the original references are CPMpy models. The industrial motivation for separating unsoundness, incompleteness, and mixed errors is discussed in Appendix C.

4 Results and Analysis

Our evaluation addresses two questions: RQ1 validates the CPJUDGEBENCH construction pipeline, and RQ2 studies LLM-as-a-judge effectiveness and model-selection trade-offs.

RQ1. Generator-Level Evaluation. How reliable is the solver-supported data construction pipeline? We measure whether label-conditioned generation followed by solver validation can popu-

Table 2: Data statistics of CPJUDGEBENCH by CP language and oracle semantic label. Each cell reports count and within-language percentage.

Stat.	CPMpy	MiniZinc	Overall
Families	82	82	82
Models	**328**	**194**	**522**
EQ	60 (18.3)	28 (14.4)	88 (16.9)
UNS	55 (16.8)	22 (11.3)	77 (14.8)
INC	76 (23.2)	38 (19.6)	114 (21.8)
MIX	60 (18.3)	24 (12.4)	84 (16.1)
NONEXEC	77 (23.5)	82 (42.3)	159 (30.5)

late the intended semantic classes across languages, labels, problem families, and feedback attempts.

RQ2a. CP Semantic Judge Meta-Evaluation. How effectively do LLM judges recover CP solution-space semantics when their outputs are evaluated against a solver-supported oracle? We compare multi-class semantic-label prediction, a concise binary-correctness diagnostic, and score-based semantic-similarity judgments.

RQ2b. Judge Selection and Deployment Trade-offs. Which LLM judges work best, and what trade-offs should guide their use? We compare model families, scale, access type, reasoning support, coding orientation, and token prices, using the MiniZinc reference-free setting as a challenging test case.

RQ1: Generator-Level Evaluation. As shown in Stage 1 of Figure 1, the generator proposes a candidate M_t^c, but a case is retained only when the solver-supported oracle $\mathcal{O}(M_t^c, M_q^\star, V_q)$ validates its label by comparing S^c and $S^\star$ through $\mathrm{FP} = S^c \setminus S^\star$ and $\mathrm{FN} = S^\star \setminus S^c$. Table 3 therefore measures a stricter property than prompt compliance: it measures whether the generated model actually realizes the intended solution-space relation.

Overall, 522 of 820 target cases are retained, corresponding to a 63.7% success rate, and 357 retained cases are found on the first attempt. The language gap is substantial: CPMpy succeeds in 80.0% of target cases, whereas MiniZinc succeeds in 47.3%. This gap is consistent with the benchmark design: the original references are CPMpy models, while MiniZinc candidates require cross-language reinterpretation and are more prone to syntax and modeling mismatches.

The label-level results show that NONEXEC is the easiest class to generate, with success rates of 93.9% for CPMpy and 100.0% for MiniZinc. In contrast, labels that require controlled semantic distortion are much harder, especially in MiniZinc: UNS reaches only 26.8%, MIX 29.3%, and EQ 34.1%. The MiniZinc median attempt count is 5.0 for all executable semantic labels, meaning that at least half of those target cases reach the maximum feedback budget. Thus, the feedback loop is useful as a construction mechanism, but the bottleneck is not merely producing code; it is producing a model whose projected solution space has the requested relation to the reference.

Table 3: Benchmark construction outcomes by language and semantic label. Each case is a problem–language–target-label combination with up to five solver-feedback attempts. Shaded cells highlight RQ1-relevant trends.

Lang.	Label	Cases	Succ.	Fail.	Rate	Tot.	Avg.	Med.	1st	Opp.
Overall		820	522	298	63.7%	2,315	2.82	2.0	357	203
CPMpy	**All**	410	328	82	80.0%	873	2.13	1.0	235	93
	EQ	82	60	22	73.2%	172	2.10	1.0	43	2
	UNS	82	55	27	67.1%	228	2.78	2.0	35	6
	INC	82	76	6	92.7%	129	1.57	1.0	59	28
	MIX	82	60	22	73.2%	216	2.63	2.0	37	28
	NONEXEC	82	77	5	93.9%	128	1.56	1.0	61	29
MiniZinc	**All**	410	194	216	47.3%	1,442	3.52	5.0	122	110
	EQ	82	28	54	34.1%	346	4.22	5.0	11	2
	UNS	82	22	60	26.8%	356	4.34	5.0	8	1
	INC	82	38	44	46.3%	306	3.73	5.0	13	18
	MIX	82	24	58	29.3%	347	4.23	5.0	11	12
	NONEXEC	82	82	0	100.0%	87	1.06	1.0	79	77

Notes: Succ./Fail. are retained/failed cases; Rate is success rate. Tot./Avg./Med. are attempt counts. 1st denotes first-attempt success. Opp. denotes valid captures with an oracle label different from the target label.

Table 4: Score-based assessment on CPMpy. Reference-based metric baselines and LLM-as-a-judge scores are compared against solver-supported oracle scores using Kendall's τ, Pearson's r_p, and Spearman's r_s.

Method	n	$\tau \uparrow$	$r_p \uparrow$	$r_s \uparrow$
Reference-based metric baselines				
BLEU	328	-0.039	-0.060	-0.056
ROUGE-L	328	-0.093	-0.110	-0.125
CodeBLEU	328	-0.035	-0.066	-0.054
METEOR	328	**0.022**	**0.050**	**0.027**
chrF	328	0.008	0.003	0.006
RUBY	328	0.006	0.047	0.008
CodeBERTScore-F1	328	-0.077	-0.085	-0.105
CodeBERTScore-F3	328	-0.010	-0.015	-0.014
LLM-as-a-judge score results				
Gemini 2.5 Pro	273	**0.382**	**0.453**	**0.421**
Claude Sonnet 4.6	274	0.331	0.421	0.382
Qwen3-32B	274	0.277	0.339	0.317
Qwen3-Coder-Next	274	0.257	0.349	0.302
GPT-4o-mini	274	0.153	0.206	0.182
Llama 4 Scout	274	0.094	0.127	0.107

Table 5: Multi-class semantic-label accuracy for the 12 judges evaluated across both CP languages and judging settings, reported by oracle label. RF and RB denote reference-free and reference-based judging. Shaded cells mark values discussed in RQ2a: green for strongest overall results, blue for notable reference-based gains on CPMpy, and orange for difficult semantic regions.

Lang.	Set.	Label	Claude Sonnet 4.6	Gemini 2.5 Pro	Qwen3-32B	Qwen3-Coder-Next	Llama 4 Scout	GPT-4o-mini	DeepSeek Chat v3.1	Gemini 2.5 Flash	Phi-4	Codestral 2508	Mistral Small 3.2	GPT-4.1-mini
CPMpy	RF	Overall	0.599	0.574	0.440	0.255	0.344	0.223	0.299	0.281	0.224	0.285	0.281	0.388
		EQ	0.768	0.696	0.607	0.143	0.679	0.054	0.316	0.123	0.123	0.632	0.333	0.667
		UNS	0.750	0.727	0.409	0.273	0.114	0.091	0.295	0.045	0.386	0.409	0.364	0.455
		INC	0.531	0.547	0.438	0.062	0.266	0.000	0.250	0.438	0.266	0.016	0.375	0.141
		MIX	0.519	0.519	0.444	0.463	0.463	0.870	0.094	0.377	0.057	0.340	0.321	0.264
		NONEXEC	0.484	0.438	0.312	0.359	0.188	0.141	0.508	0.349	0.302	0.111	0.048	0.444
	RB	Overall	0.688	0.709	0.642	0.401	0.514	0.298	0.541	0.491	0.384	0.434	0.473	0.484
		EQ	0.786	0.857	0.821	0.661	0.839	0.196	0.825	0.702	0.596	0.825	0.737	0.842
		UNS	0.864	0.864	0.795	0.386	0.364	0.227	0.591	0.295	0.568	0.523	0.523	0.364
		INC	0.750	0.797	0.688	0.219	0.500	0.000	0.547	0.453	0.219	0.156	0.500	0.469
		MIX	0.574	0.593	0.519	0.463	0.593	0.907	0.302	0.415	0.321	0.472	0.528	0.302
		NONEXEC	0.516	0.484	0.438	0.312	0.281	0.219	0.444	0.540	0.286	0.270	0.127	0.413
MiniZinc	RF	Overall	0.497	0.469	0.397	0.279	0.190	0.144	0.267	0.256	0.228	0.167	0.217	0.267
		EQ	0.889	0.741	0.741	0.444	0.741	0.074	0.481	0.407	0.370	0.741	0.444	0.704
		UNS	0.882	0.882	0.588	0.647	0.000	0.000	0.353	0.176	0.588	0.353	0.118	0.235
		INC	0.486	0.541	0.514	0.081	0.189	0.000	0.405	0.054	0.297	0.000	0.378	0.108
		MIX	0.500	0.636	0.409	0.636	0.227	0.909	0.091	0.682	0.045	0.136	0.455	0.409
		NONEXEC	0.276	0.197	0.171	0.132	0.026	0.052	0.156	0.195	0.117	0.013	0.013	0.156
	RB	Overall	0.480	0.486	0.391	0.285	0.246	0.162	0.303	0.314	0.217	0.200	0.223	0.309
		EQ	0.778	0.926	0.778	0.815	0.815	0.185	0.778	0.593	0.593	0.815	0.741	0.852
		UNS	0.941	0.941	0.647	0.765	0.412	0.176	0.471	0.176	0.471	0.235	0.176	0.588
		INC	0.432	0.595	0.541	0.054	0.216	0.000	0.143	0.400	0.086	0.000	0.143	0.314
		MIX	0.636	0.545	0.500	0.318	0.318	0.909	0.091	0.455	0.091	0.409	0.500	0.227
		NONEXEC	0.250	0.158	0.092	0.092	0.000	0.013	0.230	0.162	0.122	0.000	0.000	0.068

Answer to RQ1. The construction pipeline is reliable enough to create a diverse solver-validated benchmark, but controlled generation of CP semantic labels remains non-trivial. The oracle is essential: retained labels are grounded in solution-space comparison, not in the requested prompt label or in the generator's self-assessment.

RQ2a: CP Semantic Judge Meta-Evaluation. Stage 2 of Figure 1 compares each judge output $J_\theta(p_i^J)$ with the oracle label $y_i^\star$ or score $s_i^\star$. Table 5 focuses on the most informative setting: multi-class semantic-label prediction, where $\mathrm{Acc}_{\mathrm{multi}} = N^{-1}\sum_i \mathbb{I}[\hat{y}_i = y_i^\star]$. This setting asks not only whether a candidate is incorrect, but also how it is incorrect.

As shown in Table 5, reference-based prompting helps when the candidate and reference share a modeling language. On CPMpy, the 12-judge average increases from 0.349 in the reference-free setting to 0.505 in the reference-based setting. Statistical testing supports this trend: 11 of the 12 judge-specific gains remain significant after Bonferroni correction; for example, Claude Sonnet 4.6 improves from 0.599 to 0.688 (paired McNemar, $p = 0.00061$). The largest gains are observed for DeepSeek Chat v3.1 (0.299 to 0.541), Gemini 2.5 Flash (0.281 to 0.491), and Qwen3-32B (0.440 to 0.642). For MiniZinc, reference information is less reliable: the average changes only from 0.282 to 0.301, and the difficult NONEXEC row remains weak for most judges. Claude Sonnet 4.6 leads both reference-free settings, while Gemini 2.5 Pro leads both reference-based settings. The lower reference-free average for MiniZinc than CPMpy (0.282 vs. 0.349) further suggests that judges handle Python-like CPMpy more effectively than the more specialized MiniZinc language; however, this comparison is descriptive rather than causal because construction success and retained candidate composition differ. Appendix I provides reference-free confusion matrices for both languages and class-level diagnostics across the RF and RB settings, including false EQ acceptance and prediction-collapse patterns.

Binary correctness is useful only as a coarse diagnostic. For the six judges included in the binary analysis, collapsing all non-EQ labels into a single incorrect class produces much higher averages than semantic-label prediction: 0.764 vs. 0.406 on CPMpy and 0.729 vs. 0.329 on MiniZinc. However, this should not be interpreted as a strong binary judgment. Since EQ is a minority class, an always-INCORRECT majority baseline is already high; Appendix H reports these baselines and the compact binary summary. For this reason, the main analysis prioritizes semantic-label and score-based meta-evaluation.

Table 4 adds the graded view based on the oracle score $s^\star = 2PR/(P+R)$. Standard text and code metrics correlate weakly or negatively with the oracle score; for example, METEOR is the strongest metric baseline by Pearson correlation but reaches only 0.050. LLM score judgments are substantially better, led by Gemini 2.5 Pro ($\tau = 0.382$, $r_p = 0.453$, $r_s = 0.421$) and Claude Sonnet 4.6 (0.331, 0.421, 0.382). This indicates that strong LLM judges capture graded CP semantic similarity

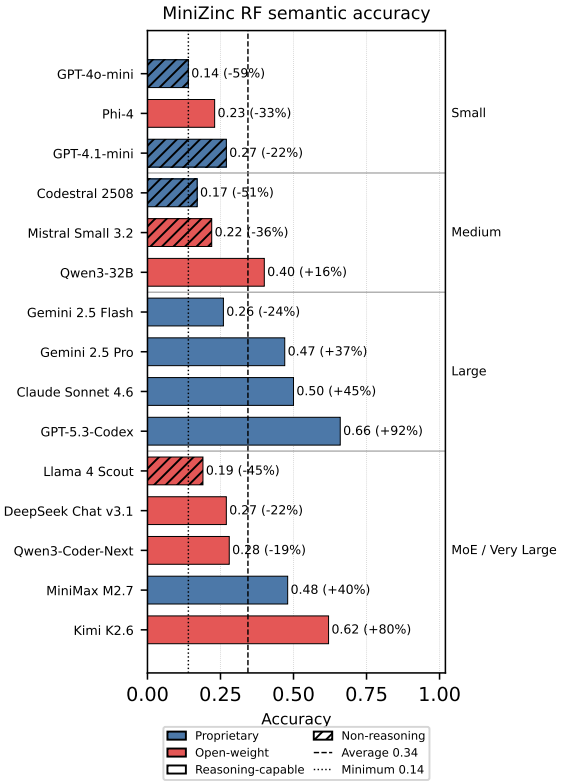

Figure 2: Reference-free semantic accuracy on MiniZinc by LLM judge. Labels show accuracy and change relative to the 15-model average; reference lines mark the average and weakest baseline. Colors indicate access type, and hatching marks non-reasoning models.

better than surface-level reference metrics, even though their categorical labels remain imperfect.

Answer to RQ2a. LLM judges can provide useful CP semantic judgments, especially in same-language reference-based CPMpy settings and in score-based assessment. However, binary accuracy overstates reliability because it collapses distinct error types and is affected by class imbalance. Robust CP judge meta-evaluation should therefore remain grounded in solver-supported semantic labels and solution-space scores.

RQ2b: Judge Selection and Deployment Trade-offs. Figure 2 isolates the hardest broad setting in our experiments: semantic-label prediction for MiniZinc without a reference model. Accuracy varies from 0.14 for GPT-4o-mini to 0.66 for GPT-5.3-Codex, with a 15-model average of 0.344. The strongest model is therefore 371% above the weakest judge and 92% above the average. The best open-weight MoE model, Kimi K2.6, reaches 0.62, which is 343% above the weakest judge and 80% above the average. At the group level, reasoning-capable models average 0.417, while non-reasoning models average 0.198. This corresponds to a 111% relative improvement, although reasoning support is confounded with model scale, provider, and coding capability.

Appendix E combines the MiniZinc reference-free accuracies from Figure 2 with the pricing metadata from Appendix D. GPT-5.3-Codex achieves the best absolute accuracy, though at a high listed output-token price. Kimi K2.6 offers a nearly comparable open-weight MoE result at lower listed output cost, while MiniMax M2.7 provides a cost-aware trade-off point, reaching 0.48 accuracy, 40% above the 15-model average. Qwen3-32B is the strongest medium model and is inexpensive, but its 0.40 accuracy suggests use for triage or ensemble pre-filtering rather than final semantic adjudication.

Answer to RQ2b. Stronger reasoning and coding-oriented judges substantially reduce the weakness of LLM-as-a-judge on MiniZinc, but the best choice depends on the deployment goal. The strongest proprietary reasoning models are most appropriate for final semantic audits, while lower-cost medium or MoE models are more practical for large-scale screening. In all cases, binary-only judgments should be avoided when the distinction between unsoundness and incompleteness matters.

5 Conclusion and Future Directions

We introduced CPJUDGEBENCH, a solver-supported benchmark for meta-evaluating LLM judges in constraint programming. The benchmark separates candidate generation from judge evaluation, assigns oracle labels through projected solution-space comparison, and evaluates whether judges can recover equivalence, unsoundness, incompleteness, mixed errors, and non-executability.

Our results show that CP provides a principled testbed for LLM-as-a-judge evaluation: the pipeline produces diverse solver-validated data, but controlled semantic generation remains challenging; LLM judges align better with solution-space scores than surface text/code metrics, while semantic-label prediction remains difficult, particularly in the MiniZinc reference-free setting. Future work should scale oracle construction beyond exhaustive enumeration, extend the oracle to objective-aware optimization semantics, and incorporate larger industrial CP models with richer domain constraints, human modeling preferences, and multilingual or multi-paradigm settings.

Limitations

Scale of the semantic oracle. CPJUDGEBENCH focuses on finite CP instances for which complete solution-space comparison is feasible under the benchmark resource limits. This choice is essential for obtaining reliable oracle labels, but it restricts the benchmark to relatively compact instances. The benchmark should therefore be understood as a controlled exact meta-evaluation protocol rather than an industrial-scale deployment. Larger industrial models may have solution spaces that cannot be exhaustively enumerated. In such settings, solver-supported validation may require symbolic reasoning, abstraction, sampling, decomposition, or counterexample-guided comparison rather than full enumeration. Also, all 82 problem families are drawn from DCP-Bench-Open (Michailidis et al., 2026), so generalizability to other CP datasets and application domains remains untested.

Feasible-space rather than objective-aware semantics. The current oracle compares projected feasible spaces. This is appropriate for constraint satisfaction and for many configuration-style settings, but it does not fully capture optimization semantics. Two optimization models can have the same feasible set but different objectives, different optimal assignments, or different objective values. Objective-aware extensions should compare projected optimal solution sets, objective values, or dominance relations where appropriate.

Dependence on references and semantic variables. The oracle assumes that the reference model is correct and that the semantic projection variables V_q capture the intended comparison space. If a reference model is incomplete, contains a modeling error, or omits relevant semantic variables, then the oracle labels inherit that limitation. Similarly, an overly narrow projection can hide semantic differences involving auxiliary or derived quantities, while an overly broad projection can penalize benign representational choices. Accordingly, the study measures agreement with solver-supported oracle labels rather than expert-level performance; no independent expert audit of the reference models or semantic projections was conducted.

Synthetic candidate distribution. Candidate models are produced through label-conditioned generation rather than collected from human modelers. This makes it possible to populate rare semantic error classes, but it may introduce artifacts from the generator model, prompt templates, feedback loop, or selected CP languages. Naturally occurring modeling mistakes may differ in style, locality, and severity. Future work should combine controlled synthesis with human-written or industrial candidate models. MiniZinc's lower construction success, particularly for UNS and MIX, may introduce selection effects that stratification cannot fully remove; cross-language comparisons are therefore descriptive rather than causal. Using a single fixed generator also leaves multi-generator and cross-generator generalizability outside the present scope.

Judge and prompt dependence. The reported judge performance depends on the selected models, prompt templates, decoding settings, output parsers, and pricing metadata. Since commercial and open-weight LLMs are updated frequently, absolute performance and cost trade-offs may change. The benchmark should therefore be interpreted as a reproducible evaluation protocol and dataset rather than a permanent ranking of model providers. Because each candidate–setting pair receives one judgment, run-to-run sampling variability is not estimated.

Binary correctness is intrinsically limited. Binary correctness collapses all non-equivalent labels into a single incorrect class. This can be useful for coarse filtering, but it is insufficient for CP modeling analysis because unsoundness, incompleteness, mixed errors, and non-executability have different practical implications. The class imbalance induced by this collapse also makes binary accuracy easy to overinterpret. For this reason, semantic-label and score-based results are the primary evidence in the paper.

Acknowledgments

This work was supported by the Independent Research Fund Denmark (DFF), grant ID 10.46540/3163-00006B.

AI Assistance Statement

We used AI assistants in a limited and supportive capacity during the preparation of this work. Specifically, AI tools were used for assistance with coding/debugging, grammar checking, minor proofreading, and support in preparing or refining visual materials. All scientific content, analysis, interpretations, claims, and final decisions were made and

verified by the authors. No AI tool was used as an author, and all AI-assisted outputs were reviewed and edited by the authors before inclusion.

References

Satanjeev Banerjee and Alon Lavie. 2005. Meteor: An automatic metric for mt evaluation with improved correlation with human judgments. In *Proceedings of the acl workshop on intrinsic and extrinsic evaluation measures for machine translation and/or summarization*, pages 65–72.

Anna Bavaresco, Raffaella Bernardi, Leonardo Bertolazzi, Desmond Elliott, Raquel Fernández, Albert Gatt, Esam Ghaleb, Mario Giulianelli, Michael Hanna, Alexander Koller, and 1 others. 2025. Llms instead of human judges? a large scale empirical study across 20 nlp evaluation tasks. In *Proceedings of the 63rd Annual Meeting of the Association for Computational Linguistics (Volume 2: Short Papers)*, pages 238–255.

Mark Chen, Jerry Tworek, Heewoo Jun, Qiming Yuan, Henrique Ponde De Oliveira Pinto, Jared Kaplan, Harri Edwards, Yuri Burda, Nicholas Joseph, Greg Brockman, and 1 others. 2021. Evaluating large language models trained on code. *arXiv preprint arXiv:2107.03374*.

Xiaoyun Fu, Haoyu Zhang, Liting Jing, Xiaoyan Fan, Congda Lu, and Shaofei Jiang. 2024. A constraint-driven conceptual design approach for product based on function-behavior-structure design process. *Computers & Industrial Engineering*, 189:109994.

Tias Guns. 2019. Increasing modeling language convenience with a universal n-dimensional array, cppy as python-embedded example. In *Proceedings of the 18th workshop on Constraint Modelling and Reformulation at CP (Modref 2019)*, volume 19.

Hongchao Jiang, Yiming Chen, Yushi Cao, Hung yi Lee, and Robby T. Tan. 2025. Codejudgebench: Benchmarking llm-as-a-judge for coding tasks. *arXiv preprint arXiv: 2507.10535*.

Christophe Lecoutre and Nicolas Szczepanski. 2020. Pycsp3: modeling combinatorial constrained problems in python. *arXiv preprint arXiv:2009.00326*.

Chin-Yew Lin. 2004. Rouge: A package for automatic evaluation of summaries. In *Text summarization branches out*, pages 74–81.

Kostis Michailidis, Dimos Tsouros, and Tias Guns. 2025. Cp-bench: Evaluating large language models for constraint modelling. *arXiv preprint arXiv:2506.06052*.

Kostis Michailidis, Dimos Tsouros, and Tias Guns. 2026. Dcp-bench-open: Evaluating llms for constraint modelling of discrete combinatorial problems. *Preprint*, arXiv:2506.06052.

Nicholas Nethercote, Peter J Stuckey, Ralph Becket, Sebastian Brand, Gregory J Duck, and Guido Tack. 2007. Minizinc: Towards a standard cp modelling language. In *International conference on principles and practice of constraint programming*, pages 529–543. Springer.

Zhiwei Pan, Zili Wang, Lemiao Qiu, Shuyou Zhang, Hong Zhu, Huang Zhang, Feifan Xiang, and Changlong Cheng. 2025. ConfigRec: An efficient recommendatory configuration design method for customized products. *Journal of Manufacturing Systems*, 82:161–177.

Kishore Papineni, Salim Roukos, Todd Ward, and Wei-Jing Zhu. 2002. Bleu: a method for automatic evaluation of machine translation. In *Proceedings of the 40th annual meeting of the Association for Computational Linguistics*, pages 311–318.

Roberto Penco, Damir Pintar, Mihaela Vranić, and Marko Šoštarić. 2025. Large language model-driven framework for automated constraint model generation in configuration problems. *Applied Sciences*, 15(12):6518.

Maja Popović. 2015. chrf: character n-gram f-score for automatic mt evaluation. In *Proceedings of the tenth workshop on statistical machine translation*, pages 392–395.

Shuo Ren, Daya Guo, Shuai Lu, Long Zhou, Shujie Liu, Duyu Tang, Neel Sundaresan, Ming Zhou, Ambrosio Blanco, and Shuai Ma. 2020. Codebleu: a method for automatic evaluation of code synthesis. *arXiv preprint arXiv:2009.10297*.

Sara Shafiee, Anders Haug, Saeedeh Shafiee Kristensen, and Lars Hvam. 2021. Application of design thinking to product-configuration projects. *Journal of Manufacturing Technology Management*, 32(1):219–241.

Sara Shafiee, Katrin Kristjansdottir, Lars Hvam, and Cipriano Forza. 2018. How to scope configuration projects and manage the knowledge they require. *Journal of Knowledge Management*, 22(5):982–1014.

Doaa Abdelaziz Shawky, Julien Le Duigou, and Joanna Daaboul. 2026. Product configuration for mass customisation: A systematic literature review. *International Journal of Production Research*, pages 1–33.

Weichun Shi, Minghao Liu, Wanting Zhang, Langchen Shi, Fuqi Jia, Feifei Ma, and Jian Zhang. 2025. Constraintllm: A neuro-symbolic framework for industrial-level constraint programming. In *Proceedings of the 2025 Conference on Empirical Methods in Natural Language Processing*, pages 16010–16030.

Yuliang Song and Eldan Cohen. 2025. Do llms understand constraint programming? zero-shot constraint programming model generation using llms. In *International Conference on Learning and Intelligent Optimization*, pages 16–31. Springer.

Sijun Tan, Siyuan Zhuang, Kyle Montgomery, William Y Tang, Alejandro Cuadron, Chenguang Wang, Raluca Ada Popa, and Ion Stoica. 2024. Judgebench: A benchmark for evaluating llm-based judges. *arXiv preprint arXiv:2410.12784*.

Weixi Tong and Tianyi Zhang. 2024. Codejudge: Evaluating code generation with large language models. In *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*, pages 20032–20051.

Ngoc Tran, Hieu Tran, Son Nguyen, Hoan Nguyen, and Tien Nguyen. 2019. Does bleu score work for code migration? In *2019 IEEE/ACM 27th International Conference on Program Comprehension (ICPC)*, pages 165–176. IEEE.

Xinwei Zhang, Qiong Feng, Yihang Li, Chen Zheng, and Salvatore Corrente. 2025. A representative product configuration ranking approach considering requirement interactions and inconsistent group preferences. *International Journal of Production Economics*, 282:109534.

Yuwei Zhao, Ziyang Luo, Yuchen Tian, Hongzhan Lin, Weixiang Yan, Annan Li, and Jing Ma. 2025. Codejudge-eval: Can large language models be good judges in code understanding? In *Proceedings of the 31st International Conference on Computational Linguistics*, pages 73–95.

Shuyan Zhou, Uri Alon, Sumit Agarwal, and Graham Neubig. 2023. Codebertscore: Evaluating code generation with pretrained models of code. In *Proceedings of the 2023 Conference on Empirical Methods in Natural Language Processing*, pages 13921–13937.

Terry Yue Zhuo. 2024. Ice-score: Instructing large language models to evaluate code. In *Findings of the association for computational linguistics: EACL 2024*, pages 2232–2242.

Terry Yue Zhuo, Minh Chien Vu, Jenny Chim, Han Hu, Wenhao Yu, Ratnadira Widyasari, Imam Nur Bani Yusuf, Haolan Zhan, Junda He, Indraneil Paul, and 1 others. 2025. Bigcodebench: Benchmarking code generation with diverse function calls and complex instructions. In *International Conference on Learning Representations 2025*, pages 99488–99542. OpenReview.

Appendix Overview

The appendix contains reproducibility and supplementary-analysis material that supports the main paper without interrupting the main argument. Appendix A illustrates the solution-space labels with a toy example. Appendix B presents the complete benchmark workflow, including the iterative candidate-generation and validation loop and the LLM-judge configurations. Appendix C motivates the semantic labels from product-configuration and design settings. Appendix D reports the LLM judge metadata and experiment-time pricing, and Appendix E gives the corresponding accuracy–cost trade-off table. Appendix F gives the label-conditioned generation prompts used in Stage 1. Appendix G gives the judging prompts used in Stage 2. Appendix H reports the binary-correctness summary that is intentionally kept out of the main paper because binary accuracy is coarse and class-imbalanced.

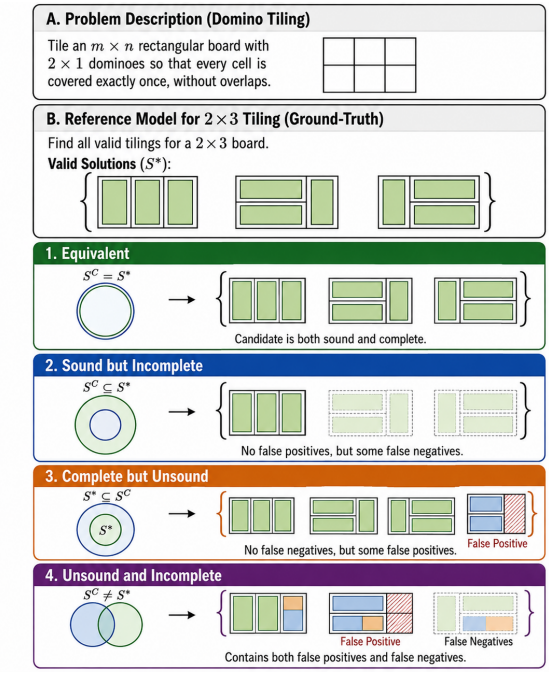

Figure 3: Solution-space-based correctness labels illustrated with a 2×3 domino-tiling toy problem. Candidate models are labeled by the relation between their projected solution space S^c and the reference solution space $S^\star$.

A Illustration of Solution-Space Labels

Figure 3 illustrates the semantic labels used in CPJUDGEBENCH with a small 2×3 domino-tiling example. The labels are determined by the relation between the projected candidate solution space S^c and the reference solution space $S^\star$: EQ denotes equality, UNS denotes extra invalid assignments, INC denotes missing valid assignments, and MIX denotes both error types.

B Detailed framework

Figure 4 provides a more detailed representation of the CPJUDGEBENCH framework, including the candidate-generation inputs, iterative solver-validation process, feedback mechanism, judge

configurations, and meta-evaluation outputs.

C Industrial Motivation and Label Design

Product configuration and engineering design provide a practical motivation for solution-space labels. In these settings, a CP model does not merely need to find one feasible configuration; it must represent the set of admissible product variants that customers, engineers, or downstream systems may explore. This makes the distinction between unsoundness and incompleteness operationally important. An unsound model can expose invalid configurations that cannot be manufactured, delivered, or safely combined. An incomplete model can remove valid choices, unnecessarily narrowing the design space. A mixed model combines both risks.

This motivation supports the five-label design used in CPJUDGEBENCH. EQ captures semantic equivalence, UNS captures false positives, INC captures false negatives, MIX captures both types of semantic error, and NONEXEC captures failures before solution-space comparison is possible. The labels are therefore more informative than a binary correct/incorrect decision and better aligned with CP modeling practice, where different modeling errors have different downstream consequences.

D Judge Model Details

Table 6 summarizes the LLM judges used in the experiments. The selected models cover commercial and open-weight systems, general-purpose and code-specialized models, dense and mixture-of-experts architectures, and different levels of reasoning support. The price columns report USD per one million input and output tokens at the time of the experiments. They are used only as deployment metadata for the trade-off analysis in Table 7; prices may change over time and should not be interpreted as fixed long-term costs.

E Accuracy–Cost Trade-offs

Table 7 reports the accuracy–cost trade-offs for selected judges in the MiniZinc reference-free semantic-label setting. Accuracy values correspond to Figure 2, and prices follow the experiment-time metadata reported in Appendix D.

F Data Generation Prompt Templates

This appendix reports the label-conditioned prompt templates used to generate candidate CP models. Each prompt receives a problem description, a reference model, an example instance, a target CP language, and a target semantic label. The target label is used only to guide generation. The solver-supported oracle assigns the final benchmark label by comparing projected solution spaces, so a generated candidate is retained only if the oracle validates the intended semantic relation or if it is captured under a valid opportunistic label.

F.1 Equivalent Candidate Generation

Generation prompt for EQ candidates

Generate one candidate CP model for this problem.
Problem:
`{problem_description}`
Reference model (for understanding only):
`{reference_cp_model}`
Example instance:
`{example_instance}`
Language: `{language}`
Target label: EQ
Target behavior: `{label_guidance}`
Before writing code, reason through these steps:

1. **COMPREHEND** – Identify decision variables, all hard constraints, including implicit ones, and feasibility semantics.
2. **DIVERGE** – Choose a structurally distinct reformulation: change variable type or perspective, swap constraint families, or adopt a dual viewpoint. Cosmetic rewrites do not qualify.
3. **AUDIT** – Confirm every reference constraint is semantically covered and no unintended restrictions are added. Verify objectives evaluate identically on all feasible assignments.
4. **GATE** – Only output code if both hold: the formulation is structurally distinct and the solution space is provably identical.

Practical requirements:

- Prefer propagation-efficient encodings.
- Preserve all expected instance fields with correct types.
- Output solutions using the same variable names and structure as the reference model so solutions can be directly compared.

`{feedback_section}{previous_attempt_section}`
Output only raw code. Use Python for CPMpy/pyCSP3 and `.mzn` text for MiniZinc. Place the reformulation strategy as a single-line comment at the top, using `%` for MiniZinc and # for Python.

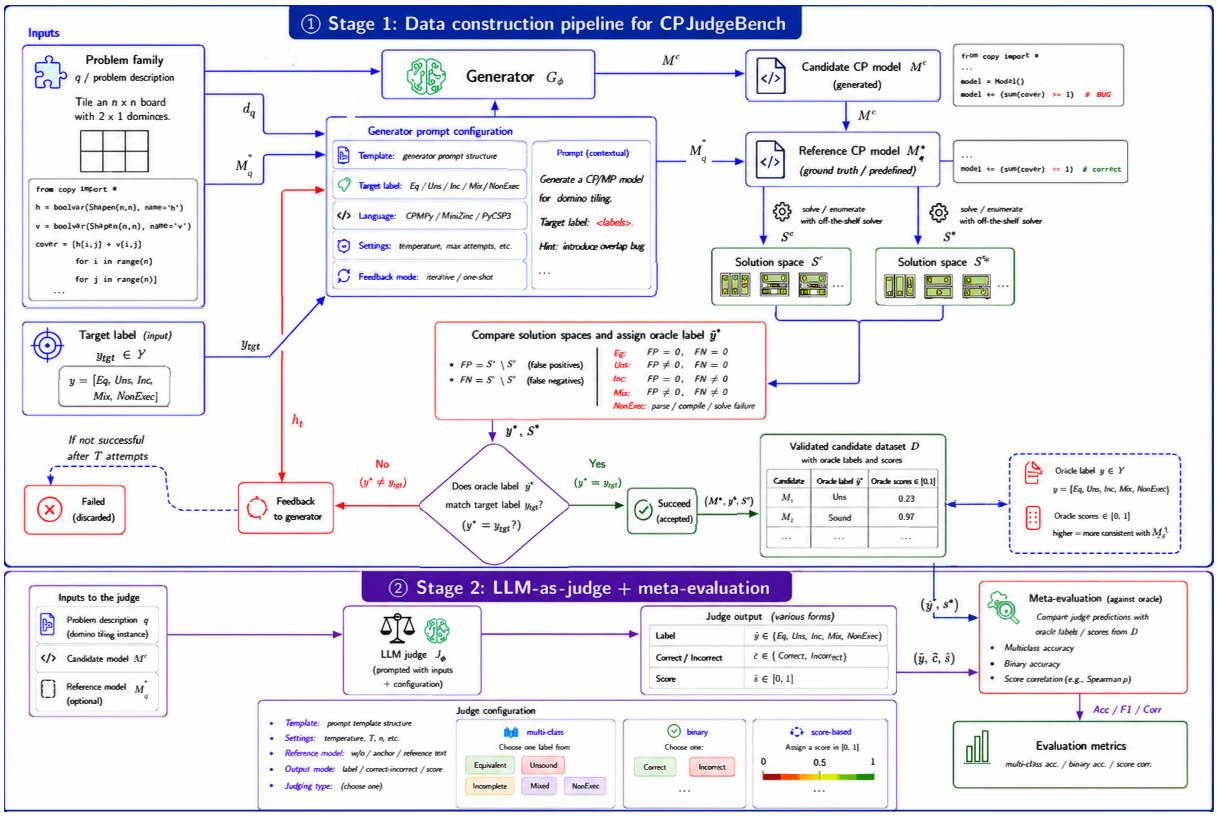

Figure 4: Detailed framework of CPJUDGEBENCH. Stage 1 generates candidate CP models using the problem description, reference model, target language, and requested semantic label. The candidates are iteratively validated through solver-supported projected solution-space comparison, with validation feedback used for subsequent generation attempts. Stage 2 evaluates LLM judges under different prompting settings by comparing their semantic-label, binary-correctness, or score predictions with the oracle outputs.

F.2 Unsound Candidate Generation

Generation prompt for UNS candidates

Generate one candidate CP model for this problem.
Problem:
`{problem_description}`
Reference model (for understanding only):
`{reference_cp_model}`
Example instance:
`{example_instance}`
Language: `{language}`
Target label: UNS
Target behavior: `{label_guidance}`
Before writing code, reason through these steps:

1. **COMPREHEND** – Identify decision variables, all hard constraints, including implicit ones, and feasibility semantics.
2. **TARGET** – Identify a specific set of invalid solutions to admit. Choose one meaningful unsoundness strategy, such as relaxing a domain, weakening or dropping a necessary constraint, loosening a bound, or replacing a strict condition with a partial one.
3. **AUDIT** – Confirm the model is otherwise complete: no valid solutions are excluded, and the unsoundness is traceable to exactly one deliberate change.
4. **GATE** – Only output code if at least one invalid solution is provably admitted and no valid solutions are excluded.

Practical requirements:

- Keep the unsoundness subtle but structurally clear.
- Preserve all expected instance fields with correct types.
- Output solutions using the same variable names and structure as the reference model so solutions can be directly compared.

`{feedback_section}{previous_attempt_section}`
Output only raw code. Use Python for CPMpy/pyCSP3 and `.mzn` text for MiniZinc. Place the flaw strategy as a single-line comment at the top, using % for MiniZinc and # for Python.

F.3 Incomplete Candidate Generation

Generation prompt for INC candidates

Generate one candidate CP model for this problem.
Problem:
`{problem_description}`
Reference model (for understanding only):
`{reference_cp_model}`
Example instance:

Table 6: Judge models considered in the experiments. Prices are reported in USD per one million input/output tokens according to OpenRouter at the time of the experiments.

Model	Provider	Access	Orientation	Reasoning	Params.	Input / 1M	Output / 1M
			Small models				
GPT-4o-mini	OpenAI	Proprietary	General	Non-reasoning	Undisclosed	$0.15	$0.60
GPT-4.1-mini	OpenAI	Proprietary	General / Coding	Non-reasoning	Undisclosed	$0.40	$1.60
Phi-4	Microsoft	Open-weight	General	Reasoning-capable	14B	$0.065	$0.14
			Medium models				
Mistral Small 3.2	Mistral AI	Open-weight	General / Coding	Non-reasoning	24B	$0.075	$0.20
Qwen3-32B	Qwen	Open-weight	General / Coding	Reasoning-capable	32.8B	$0.08	$0.28
Codestral 2508	Mistral AI	Proprietary	Code	Non-reasoning	Undisclosed	$0.30	$0.90
			Large models				
Gemini 2.5 Flash	Google	Proprietary	General	Reasoning-capable	Undisclosed	$0.30	$2.50
Claude Sonnet 4.6	Anthropic	Proprietary	General / Coding	Reasoning-capable	Undisclosed	$3.00	$15.00
Gemini 2.5 Pro	Google	Proprietary	General / Coding	Reasoning-capable	Undisclosed	$1.25	$10.00
GPT-5.3-Codex	OpenAI	Proprietary	Code	Reasoning-capable	Undisclosed	$1.75	$14.00
			MoE / very large models				
Llama 4 Scout	Meta	Open-weight	General	Non-reasoning	109B total / 17B active	$0.08	$0.30
DeepSeek Chat v3.1	DeepSeek	Open-weight	General / Coding	Reasoning-capable	671B total / 37B active	$0.21	$0.79
Qwen3-Coder-Next	Qwen	Open-weight	Code	Reasoning-capable	80B total / 3B active	$0.11	$0.80
MiniMax M2.7	MiniMax	Proprietary	Agentic	Reasoning-capable	230B total / 10B active	$0.279	$1.20
Kimi K2.6	MoonshotAI	Open-weight	General / Coding	Reasoning-capable	1T total / 32B active	$0.73	$3.49

Table 7: Accuracy–cost trade-offs for selected judges. Accuracy is MiniZinc reference-free semantic-label accuracy. $\Delta_{\min}$ is relative improvement over GPT-4o-mini (0.14), and Δ_{avg} is relative improvement over the 15-model average (0.344). Prices are USD per one million input/output tokens recorded at experiment time; see Appendix D.

Role	Model	Acc.	$\Delta_{\min}$	Δ_{avg}	In	Out
Best overall	GPT-5.3-Codex	0.66	+371%	+92%	$1.75	$14.00
Best open MoE	Kimi K2.6	0.62	+343%	+80%	$0.73	$3.49
Strong general	Claude Sonnet 4.6	0.50	+257%	+45%	$3.00	$15.00
Cost-aware MoE	MiniMax M2.7	0.48	+243%	+40%	$0.279	$1.20
Score leader	Gemini 2.5 Pro	0.47	+236%	+37%	$1.25	$10.00
Best medium	Qwen3-32B	0.40	+186%	+16%	$0.08	$0.28
Weak baseline	GPT-4o-mini	0.14	–	-59%	$0.15	$0.60

`{example_instance}`
Language: `{language}`
Target label: INC
Target behavior: `{label_guidance}`
Before writing code, reason through these steps:

1. **COMPREHEND** – Identify decision variables, all hard constraints, including implicit ones, and feasibility semantics.
2. **TARGET** – Identify a specific subset of valid solutions to suppress. Choose one meaningful incompleteness strategy, such as over-restricting a domain, adding a spurious constraint, tightening a bound, or encoding only a partial case of a global constraint.
3. **AUDIT** – Confirm the model is otherwise sound: no invalid solutions are admitted, and the incompleteness is traceable to exactly one deliberate change.
4. **GATE** – Only output code if at least one valid solution is provably excluded and no invalid solutions are admitted.

Practical requirements:

- Keep the incompleteness subtle but structurally clear.
- Preserve all expected instance fields with correct types.
- Output solutions using the same variable names and structure as the reference model so solutions can be directly compared.

`{feedback_section}{previous_attempt_section}`
Output only raw code. Use Python for CPMpy/pyCSP3 and `.mzn` text for MiniZinc. Place the flaw strategy as a single-line comment at the top, using `%` for MiniZinc and `#` for Python.

F.4 Mixed Candidate Generation

Generation prompt for MIX candidates

Generate one candidate CP model for this problem.
Problem:
`{problem_description}`
Reference model (for understanding only):
`{reference_cp_model}`
Example instance:
`{example_instance}`
Language: `{language}`
Target label: MIX
Target behavior: `{label_guidance}`
Before writing code, reason through these steps:

1. **COMPREHEND** – Identify decision variables, all hard constraints, including implicit ones, and feasibility semantics.
2. **TARGET** – Introduce two independent deliberate flaws: one relaxation that admits at least one invalid solution, and one restriction that excludes at least one valid solution.
3. **AUDIT** – Confirm each flaw is traceable to exactly

one deliberate change, and that the two flaws do not accidentally cancel each other out.

4. **GATE** – Only output code if at least one invalid solution is admitted, at least one valid solution is excluded, and the two flaws are independent.

Practical requirements:

- Keep each flaw subtle but structurally clear.
- Preserve all expected instance fields with correct types.
- Output solutions using the same variable names and structure as the reference model so solutions can be directly compared.

`{feedback_section}{previous_attempt_section}`
Output only raw code. Use Python for CPMpy/pyCSP3 and `.mzn` text for MiniZinc. Place both flaw strategies as a single-line comment at the top, using `%` for MiniZinc and # for Python.

F.5 Non-executable Candidate Generation

Generation prompt for NONEXEC candidates

Generate one candidate CP model for this problem.
Problem:
`{problem_description}`
Reference model (for understanding only):
`{reference_cp_model}`
Example instance:
`{example_instance}`
Language: `{language}`
Target label: NONEXEC
Target behavior: `{label_guidance}`
Before writing code, reason through these steps:

1. **COMPREHEND** – Understand the intended model well enough to write a near-correct version.
2. **TARGET** – Introduce exactly one realistic flaw that prevents execution, such as a wrong API argument, undefined variable, invalid syntax, non-existent global constraint, or incompatible operand types.
3. **PLACE** – Embed the flaw naturally within an otherwise coherent model so it resembles a plausible modeler mistake.
4. **GATE** – Only output code if the flaw provably prevents execution and the surrounding model is otherwise structurally reasonable.

Practical requirements:

- Preserve expected instance fields and problem-aligned variable names.
- Place the flaw type as a single-line comment at the top.

`{feedback_section}{previous_attempt_section}`
Output only raw code. Use Python for CPMpy/pyCSP3 and `.mzn` text for MiniZinc.

G Judge Prompt Templates

This appendix reports the prompts used for LLM-as-a-judge evaluation. We consider three output formats: semantic-label prediction, binary correctness judgment, and score-based semantic assessment. Reference-free prompts provide the problem description and candidate model. Reference-based prompts additionally provide the reference CP model. The prompts request a structured JSON output with a short rationale, but the quantitative evaluation uses only the parsed label, verdict, or score.

G.1 Semantic Label Prediction Prompts

Reference-free semantic label prompt

You are an expert CP model judge.
Classify the candidate with exactly one allowed correctness label.
Allowed labels: EQ, UNS, INC, MIX, NONEXEC
Label guidance:

- EQ: candidate is semantically equivalent to the intended model.
- UNS: candidate admits invalid solutions.
- INC: candidate excludes valid solutions.
- MIX: candidate both admits invalid solutions and excludes valid solutions.
- NONEXEC: candidate fails parsing, compilation, or solving.

Reasoning protocol: perform internally before answering.

1. Check executability for the given language.
2. Infer intended semantics from the problem description.
3. Compare candidate constraints to intended semantics.
4. Decide the appropriate label.

Output format: strict JSON only.

```
{"label": "<one allowed label>",
 "confidence": <0..1>,
 "rationale": "<1-3 short sentences>"}
```

Problem description:
`{problem_description}`
Candidate ({language}):
`{candidate_code}`

Reference-based semantic label prompt

You are an expert CP model judge.
Classify the candidate with exactly one allowed correctness label. Use the reference model below as the ground-truth encoding of the problem.
Allowed labels: EQ, UNS, INC, MIX, NONEXEC

Label guidance:

- EQ: candidate is semantically equivalent to the reference model.
- UNS: candidate admits solutions rejected by the reference.
- INC: candidate rejects solutions accepted by the reference.
- MIX: candidate both admits rejected solutions and rejects accepted solutions.
- NONEXEC: candidate fails parsing, compilation, or solving.

Reasoning protocol: perform internally before answering.

1. Understand the reference model and the solution space it encodes.
2. Check whether the candidate is executable.
3. Compare the candidate constraints to the reference model, identifying false positives and false negatives.
4. Decide the appropriate label.

Output format: strict JSON only.

```
{"label": "<one allowed label>",
 "confidence": <0..1>,
 "rationale": "<1-3 short sentences>"}
```

Problem description:
`{problem_description}`
Reference model ({reference_language}):
`{reference_cp_model}`
Candidate ({language}):
`{candidate_code}`

G.2 Binary Correctness Prompts

Reference-free binary correctness prompt

You are an expert CP model judge.
Determine whether the candidate constraint programming model is CORRECT or INCORRECT based solely on the problem description.
Definitions:

- CORRECT: the model's solution space semantically matches what the problem requires.
- INCORRECT: the model allows invalid solutions, misses valid solutions, or cannot execute.

Output format: strict JSON only.

```
{"verdict": "correct" or "incorrect",
 "confidence": <0..1>,
 "rationale": "<1-3 short sentences>"}
```

Problem description:
`{problem_description}`
Candidate ({language}):
`{candidate_code}`

G.3 Score-Based Judgment Prompts

Reference-based score prompt

You are an expert CP model evaluator.
Score the candidate constraint programming model on a scale of 1–5 based on how closely its solution space matches the solution space of the provided reference model.
Scoring rubric:

- 5 – Equivalent solution spaces: accepts exactly the same solutions as the reference.
- 4 – Near-equivalent: very minor differences.
- 3 – Partial overlap: solution spaces overlap significantly but with notable differences.
- 2 – Poor match: solution spaces differ substantially.
- 1 – No meaningful overlap or non-executable.

Output format: strict JSON only.

```
{"score": <integer 1..5>,
 "rationale": "<1-3 short sentences>"}
```

Problem description:
`{problem_description}`
Reference model ({reference_language}):
`{reference_cp_model}`
Candidate ({language}):
`{candidate_code}`

H Binary Correctness Results

Binary correctness collapses UNS, INC, MIX, and NONEXEC into a single incorrect class. This setting is useful as a coarse diagnostic, but it is less informative than semantic-label prediction and is strongly affected by class imbalance because EQ is a minority class. Table 8 reports the reference-free binary accuracies by oracle label, while Table 9 summarizes the overall binary results against an always-INCORRECT majority baseline.

Binary accuracies are substantially higher than the corresponding semantic-label averages, but this improvement is partly explained by the class collapse. On average, the binary judges do not outperform the majority-class baseline, and the best binary result only matches this baseline for CPMpy and remains below it for MiniZinc. These results confirm that binary correctness should be interpreted only as a coarse diagnostic: it hides the distinctions between unsoundness, incompleteness, mixed semantic errors, and non-executability that are central to CP model correctness.

I Semantic-label confusion matrices

Figures 5 and 6 present the reference-free confusion matrices for CPMpy and MiniZinc. Rows rep-

Table 8: Reference-free binary-correctness accuracy by CP language and oracle label. Binary correctness treats EQ as CORRECT and all other labels as INCORRECT.

Lang.	Label	Claude Sonnet 4.6	Gemini 2.5 Pro	Qwen3-32B	Qwen3-Coder-Next	Llama 4 Scout	GPT-4o-mini
CPMpy	Overall	0.780	0.751	0.744	0.817	0.715	0.777
	EQ	0.786	0.750	0.593	0.259	0.500	0.125
	UNS	0.955	0.932	0.881	1.000	0.860	0.955
	INC	0.734	0.730	0.823	0.984	0.742	0.969
	MIX	0.889	0.889	0.830	0.981	0.830	0.944
	NONEXEC	0.609	0.531	0.629	0.871	0.683	0.891
MiniZinc	Overall	0.704	0.710	0.709	0.791	0.654	0.804
	EQ	0.926	0.926	0.731	0.385	0.556	0.259
	UNS	1.000	0.941	0.875	0.938	0.882	1.000
	INC	0.757	0.800	0.743	0.886	0.676	0.946
	MIX	0.864	0.955	0.952	0.905	0.818	1.000
	NONEXEC	0.487	0.467	0.581	0.824	0.579	0.829

Table 9: Reference-free binary-correctness summary for the six judges in Table 8. The majority baseline predicts INCORRECT for every candidate. Avg. sem. and Avg. bin. denote the average semantic-label and binary-correctness accuracy across judges, respectively.

Language	EQ share	Maj. base	Avg. sem.	Avg. bin.	Best bin.
CPMpy	18.3%	0.817	0.406	0.764	0.817
MiniZinc	14.4%	0.856	0.329	0.729	0.804

resent oracle labels and columns represent judge predictions. The CPMpy figure covers the 12 shared judges, while the MiniZinc figure includes all 15 reference-free judges.

The matrices reveal prediction collapse that is obscured by overall accuracy. Most notably, GPT-4o-mini predicts MIX for 80.1% of CPMpy cases and 78.9% of MiniZinc cases. Its strong performance on the MIX row therefore reflects a broad preference for this label rather than reliable discrimination among semantic classes. These diagnostics distinguish the consequential acceptance of incorrect candidates from broader label-collapse errors.

Also, a particularly consequential error occurs when a non-EQ candidate is predicted as EQ, because an incorrect candidate is then accepted as semantically equivalent. Aggregated across judges, this occurs for 15.2% and 17.4% of CPMpy non-EQ predictions under RF and RB judging, respectively, compared with 25.0% and 33.4% for MiniZinc. Thus, although reference access improves overall CPMpy accuracy, it does not reduce this specific false-acceptance error and increases it in both languages.

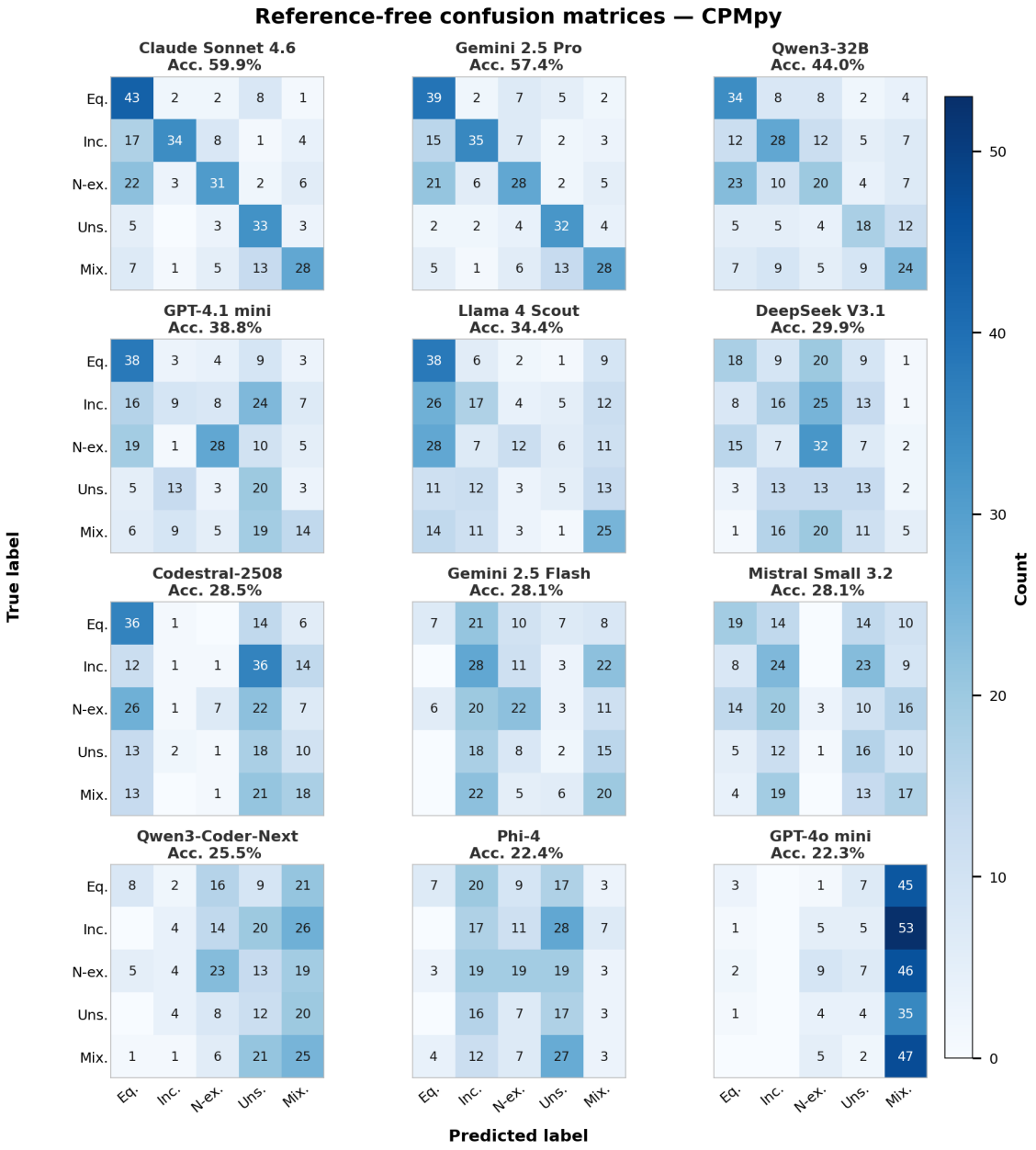

Figure 5: Reference-free semantic-label confusion matrices for CPMpy. Rows denote oracle labels and columns denote predicted labels. Each cell reports the number of predictions for the corresponding true–predicted label pair.

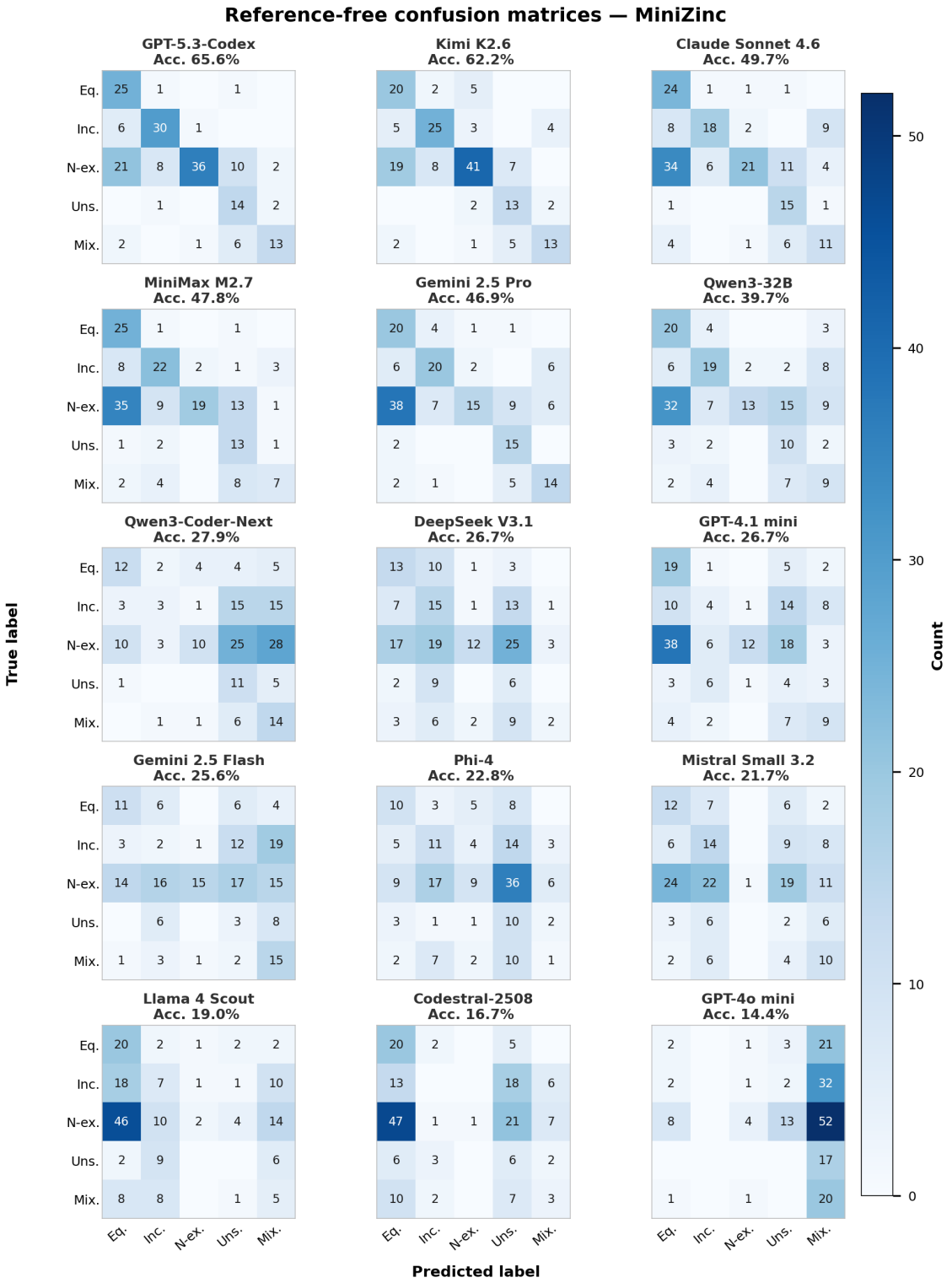

Figure 6: Reference-free semantic-label confusion matrices for MiniZinc. Rows denote oracle labels and columns denote predicted labels.